TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE
FUNDAMENTALS

Communication Architectures

(Patterns)

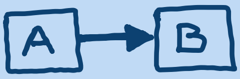
TECHNISCHE
UNIVERSITÄT
MÜNCHEN

AF 08.2
Intellectual Content: Version 1.0.3 (2020-09-26), Authored 2018-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.1 (2018-11-24), Copyright © 2018 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

UCT **Unicast** (one-to-one)

Communicate messages from one source to exactly one destination node. The destination node is explicitly and individually addressed.

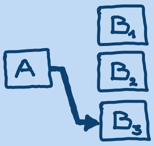
Rationale: private communication between exactly two nodes which both know each other beforehand.



ACT **Anycast** (one-to-any)

Communicate messages from one source to one of many destination nodes. The picked destination node usually is the network-topology-wise "nearest" or least utilized node in a group of nodes.

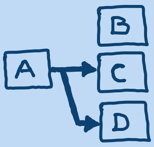
Rationale: Unicast, optimized for network failover scenarios, load balancing and CDNs.



MCT **Multicast** (one-to-many)

Communicate messages from one source to many destination nodes. The destination nodes usually form a group and are usually not individually addressed.

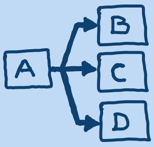
Rationale: node communication where destination nodes dynamically change or where total traffic should be reduced.



BCT **Broadcast** (one-to-all)

Communicate messages from one source to all available destination nodes. The destination nodes usually are implicitly defined by the extend of the local communication network segment.

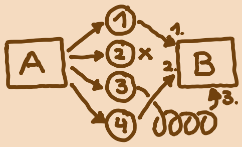
Rationale: spreading out messages to all available nodes for potential responses.



DGR **Datagram** (Single Packet)

Communicate messages as an unordered set of single packets, usually without any network congestion control, retries or other delivery guarantees.

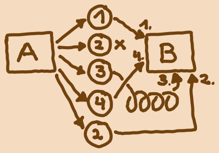
Rationale: simple low-overhead communication without prior communication establishment (handshake).



STR **Stream** (Sequence of Packets)

Communicate messages as an ordered sequence (stream) of packets, usually with network congestion control, retries and delivery guarantees (at-most-once, exactly-once, at-least-once).

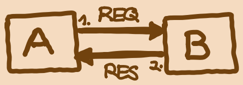
Rationale: reliable communication between nodes.



PLL **Pull** (Request/Response, RPC)

Communicate by performing a request (from the client node) and pulling a corresponding response (from the server node).

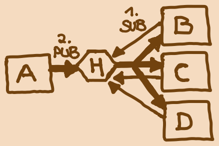
Rationale: Remote Procedure Call (RPC) like Unicast or Anycast communication.



PSH **Push** (Publish/Subscribe, Events)

Communicate by "subscribing" to "channels" of messages (on one or more receiver nodes or on an intermediate hub) once and then publishing events to those "channels" (on the sender node) multiple times.

Rationale: event-based Multicast or Broadcast communication.



The **Communication Architectures** address the kind of communication between components. One distinguishes primarily four different kinds of message transmission: with **Unicast**, a source node sends to exactly one directly addressed target node. With **Anycast**, a source node sends to a group of potential destination nodes, but the message is delivered to one destination node in the group only.

With **Multicast**, a source node also sends to a group of target nodes, but the message is delivered to all target nodes in the group. With **Broadcast**, a source node sends to all reachable destination nodes without these particular destination nodes being known to the source node.

With the kind of messages, one differentiates two variants: with **Datagram**, each message consists of exactly one network packet, and when sending, no guarantees are given whether and in which order the messages will arrive at the destination node. In contrast, with **Stream**, a message consists of a sequence of network packets and different guarantees are given: In case of packet congestion on intermediate nodes, the source of the **Stream** may be throttled. In case of packet loss, packets are resent. And one might get control over whether the packet will be delivered at most once, exactly once, or at least once at the destination node.

There are usually two modes of client/server communication: in **Pull** mode, the client sends a request, and the server sends a response. The server cannot proactively (without a prior request) send a message. In **Push** mode, the client sends a message in advance to the server to subscribe to certain types of messages. After that, the server can send a message to all subscribed clients at any time.

Usually, **Pull** is implemented via **Unicast**/**Anycast** and as a **Stream**, for example, in the HTTP protocol. On the other hand, **Push** is usually implemented via **Multicast**/**Broadcast** as a **Datagram**, for example, in the DHCP protocol.

Questions

? Which well-known Web protocol uses a communication based on **Unicast**, **Stream** and **Pull**?

Data Structure Architectures

(Patterns)

Data Structure Types

Scalar, Atom, Primitive Type

Plain integer or real number,
single character or character string,
not indexed and (for string only)
accessed in O(1) by character position.

Tuple, Object, Structural Type, Record

Ordered, fixed-size sequence of scalar elements,
each of individual type, indexed by name and
accessed in O(1) by element name.

Sequence, Array, List

Ordered sequence of elements,
each of same type, indexed by position and
accessed in O(1) or O(n) by element position.

Set, Bag, Bucket

Unordered set of elements,
each of same type, not indexed and
accessed in O(1) or O(n) by element reference.

Map, Hash, Associative Array

Unordered sequence of elements,
each of same type, indexed by (scalar) key and
accessed in O(1) by key.

Graph, Nodes & Edges

Unordered set of linked elements (nodes),
each of individual type, indexed by (scalar) key
and accessed in O(1) by key or by following a
directed link (edge).

Data Evolution Approaches

In-Place Editing

Modify data through direct in-place editing,
overwriting the previous revision.

Stacking Revisions

Modify data through stacking revisions,
preserving all previous revisions.
Latest revision is always on top of stack.

Structural Difference

Modify data through merging, journaled
domain-unspecific structural differences.

Operational Transformation (OT)

Modify data through applying journaled,
domain-specific operational transformations.

Data Sharing Approaches

Event Sourcing & CRDT

Share data as a chronological sequence of data change events from which the data states can be (re)constructed. Optionally, use a Conflict-Free Replicated Data-Type (CRDT) protocol for the change events.

Ref.-Counting & Copy-on-Write

Share data between resources by using reference-counted data chunks, duplicating a chunk (and resetting its reference count to one) on write operations only and destroying a chunk once the reference count drops to zero.

Data Store Types

Key-Value Store

Storage of values in an unordered manner,
indexed and queried by key.

Redis, Riak, MemcacheD,
RocksDB, LevelDB

Large-Object Store

Storage of unstructured binary-large object
(BLOB) data and its associated meta-data,
indexed and queried by unique id.

Minio, SeaweedFS,
AWS S3

Triple Store

Storage of subject-predicate-object triples,
indexed and queried by subject/predicate/object
values and example triples.

Redstore, Virtuoso

File-Tree Store

Storage of unstructured data as named files in a
directory tree, indexed and queried by name path
from root directory to leave file.

ZFS, XFS,
UFS2, APFS

Graph Store

Storage of values as vertices and edges in a graph, both optionally referencing associated key/value pairs. Indexed and queried by key/value pairs and traversed by following edges.

Neo4J, OrientDB, ArangoDB

Document Store

Storage of structured "documents",
indexed by id and key/value fields and
queried by id and example documents.

MongoDB, CouchDB,
RethinkDB

Relational/Table Store

Storage rows of fixed-size, typed value columns,
indexed and queried by column values.

PostgreSQL, MariaDB, SQLite,
H2, ORACLE DB, IBM DB2

Full-Text Store

Storage of unstructured text,
indexed and queried by content words.

ElasticSearch, Solr,
Groonga

Wide-Column Store

Distributed storage of rows of sparse (often untyped) value columns, indexed and queried by column values.

Cassandra, MonetDB,
HBase, ScyllaDB

Time-Series Store

InfluxDB, MetricTank,
RRDTool

Storage of integer or real values (y-axis) of a time-series (x-axis) into a fixed-size storage format in a round-robin manner where older values are increasingly aggregated (leading to lower resolutions at older times) and finally overwritten.

DataVault Store

Long-term historical storage of foreign, arbitrary relational data in a fixed schema of hubs, links and satellites, indexed and queried for analysis and reporting purposes.

DataVault 2.0

BlockChain Store

Storage of values in an unordered manner within information blocks which are cryptographically chained through their hash values and distributed in a peer-to-peer way.

Ethereum, Quorum,
Tendermint, Hyperledger

The Software Architect distinguishes only 6 **Data Structure Types** for data elements: **Scalar** (e.g. Integer, String, etc.), **Tuple** (ordered fixed-size sequence of Scalars), **Sequence** (ordered sequence of elements), **Set** (unordered set of elements), **Map** (unordered set of elements, each indexed by key) and **Graph** (unordered set of elements, each indexed by key or by following a link between elements). All complex specific data structures in practice, for the Software Architect, are only the combination of these 6 types.

There are numerous **Data Evolution Approaches**, with which data can change over time: in the simplest case, **In-Place Editing**, data is simply changed directly. Access to previous states does not exist. If one wants to access previous states, one can use **Stacking Revisions**, in which the entire data set is copied before each change. So that the entire data record does not have to be copied, **Structural Difference** stores only a technical difference between the old and the new data records. Alternatively, with **Operational Transformation**, the technical change operations can be stored as a journal.

If such a journal is used also to keep replicas of the data sets up-to-date, one refers to it as **Event Sourcing**. If the journal is used as the protocol of so-called **Conflict-Free Replicated Data-Types (CRDT)**, instead of (unidirectional) replication, a (bidirectional) synchronization can be achieved. If several processes/threads logically operate on copies, but physically on the same data sets, **Copy-on-Write** and **Reference Counting** can be used to achieve common access and the life cycle of the data sets can nevertheless be reasonably controlled.

For the storage of data in databases, there are numerous **Data Store Types**. These differ primarily in the type and flexibility of the data structure and the guarantees provided. The most common type is the **Relational/Table Store**. The most elegant type is the **Graph Store**. The most convenient is the **Document Store**.

Questions

? Name 3 **Data Evolution Approaches**, each of which allows access to the previous states of the data?

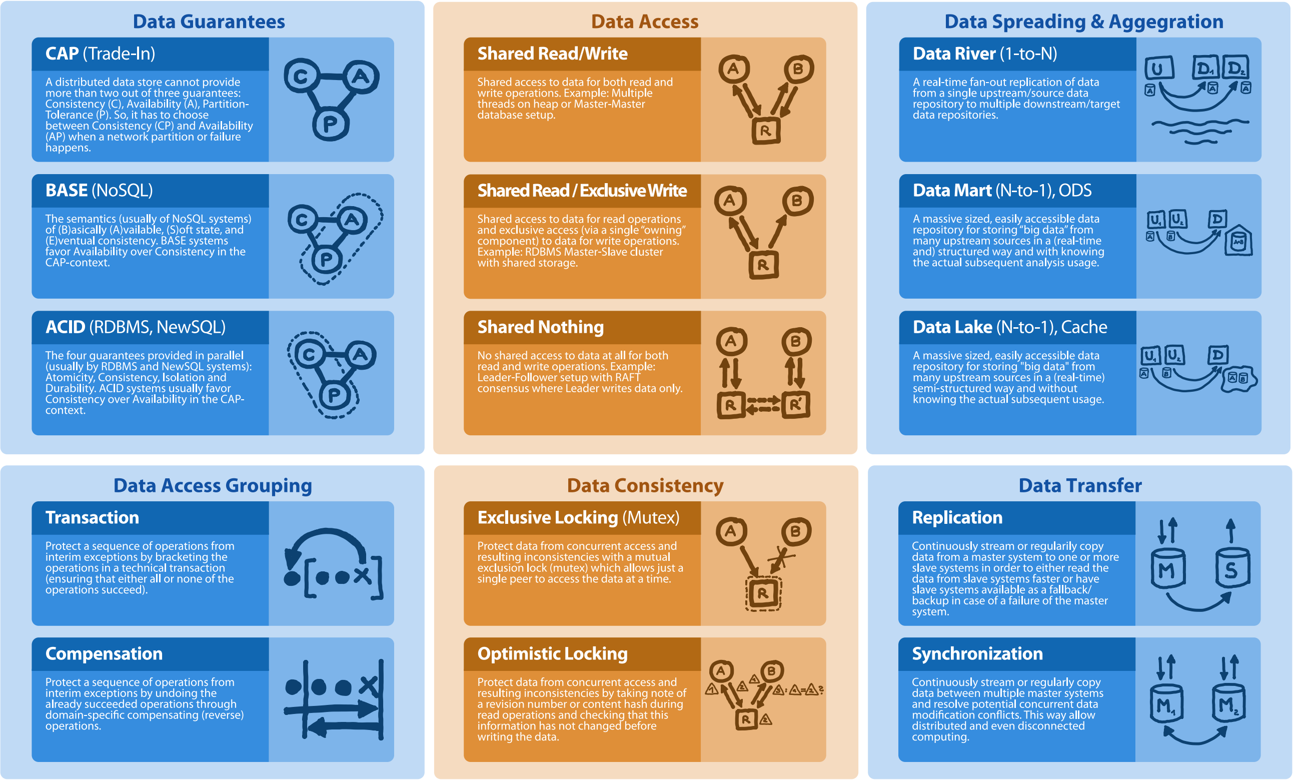
ARCHITECTURE
FUNDAMENTALS

Data Persistence Architectures

(Patterns)

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

AF 09.2



Data Guarantees

CAP (Trade-In)

A distributed data store cannot provide more than two out of three guarantees: Consistency (C), Availability (A), Partition-Tolerance (P). So, it has to choose between Consistency (CP) and Availability (AP) when a network partition or failure happens.

BASE (NoSQL)

The semantics (usually of NoSQL systems) of (B)asically (A)vailable, (S)oft state, and (E)ventual consistency. BASE systems favor Availability over Consistency in the CAP-context.

ACID (RDBMS, NewSQL)

The four guarantees provided in parallel (usually by RDBMS and NewSQL systems): Atomicity, Consistency, Isolation and Durability. ACID systems usually favor Consistency over Availability in the CAP-context.

Data Access

Shared Read/Write

Shared access to data for both read and write operations. Example: Multiple threads on heap or Master-Master database setup.

Shared Read / Exclusive Write

Shared access to data for read operations and exclusive access (via a single "owning" component) to data for write operations. Example: RDBMS Master-Slave cluster with shared storage.

Shared Nothing

No shared access to data at all for both read and write operations. Example: Leader-Follower setup with RAFT consensus where Leader writes data only.

Data Spreading & Aggegration

Data River (1-to-N)

A real-time fan-out replication of data from a single upstream/source data repository to multiple downstream/target data repositories.

Data Mart (N-to-1), ODS

A massive sized, easily accessible data repository for storing "big data" from many upstream sources in a (real-time and) structured way and with knowing the actual subsequent analysis usage.

Data Lake (N-to-1), Cache

A massive sized, easily accessible data repository for storing "big data" from many upstream sources in a (real-time) semi-structured way and without knowing the actual subsequent usage.

Data Access Grouping

Transaction

Protect a sequence of operations from interim exceptions by bracketing the operations in a technical transaction (ensuring that either all or none of the operations succeed).

Compensation

Protect a sequence of operations from interim exceptions by undoing the already succeeded operations through domain-specific compensating (reverse) operations.

Data Consistency

Exclusive Locking (Mutex)

Protect data from concurrent access and resulting inconsistencies with a mutual exclusion lock (mutex) which allows just a single peer to access the data at a time.

Optimistic Locking

Protect data from concurrent access and resulting inconsistencies by taking note of a revision number or content hash during read operations and checking that this information has not changed before writing the data.

Data Transfer

Replication

Continuously stream or regularily copy data from a master system to one or more slave systems in order to either read the data from slave systems faster or have slave systems available as a fallback/backup in case of a failure of the master system.

Synchronization

Continuously stream or regularly copy data between multiple master systems and resolve potential concurrent data modification conflicts. This way allow distributed and even disconnected computing.

In the area of **Data Guarantees**, there are three main aspects: The **CAP** theorem addresses the so-called "trade-off": In practice, one usually has to choose between Consistency + Partition-Tolerance (CP) or Availability + Partition-Tolerance (AP). Both at the same time is not possible. With **BASE** systems, AP is usually favored. For a traditional RDBMS with **ACID** guarantees, one usually favors CP.

With **Data Access Grouping** one knows about **Transaction** and **Compensation**. The former is a "technical bracket" that allows you to revert to the previous state in case of an error. The latter is a "domain-specific bracket," where so-called compensation operations allow one to "cancel" the earlier changes in order to regain a previous consistent state.

With the **Data Access** of two or more processes/threads on the same data one distinguishes between the approaches **Shared Read/Write** (all read and write the same data), **Shared Read/Exclusive Write** (all read and only one writes the same data) and **Shared Nothing** (all read and write to the equal synchronized data).

With the **Data Consistency** one knows about **Exclusive Locking** (per time unit only one writes) and **Optimistic Locking** (all try to write, but recognize and resolve a conflict).

With **Data Spreading & Aggregation** one differentiates three kinds: with the **Data River** the data are replicated from a master system to many slave systems to achieve, among other things, a higher read performance. With the **Data Mart** (structured data) and **Data Lake** (semi-structured data), data is replicated from one master system to many slave systems in order to centrally report or cache the data.

With the **Data Transfer** we finally distinguish between the unidirectional and conflict-free **Replication** and the bidirectional and conflict-rich **Synchronization**.

Questions

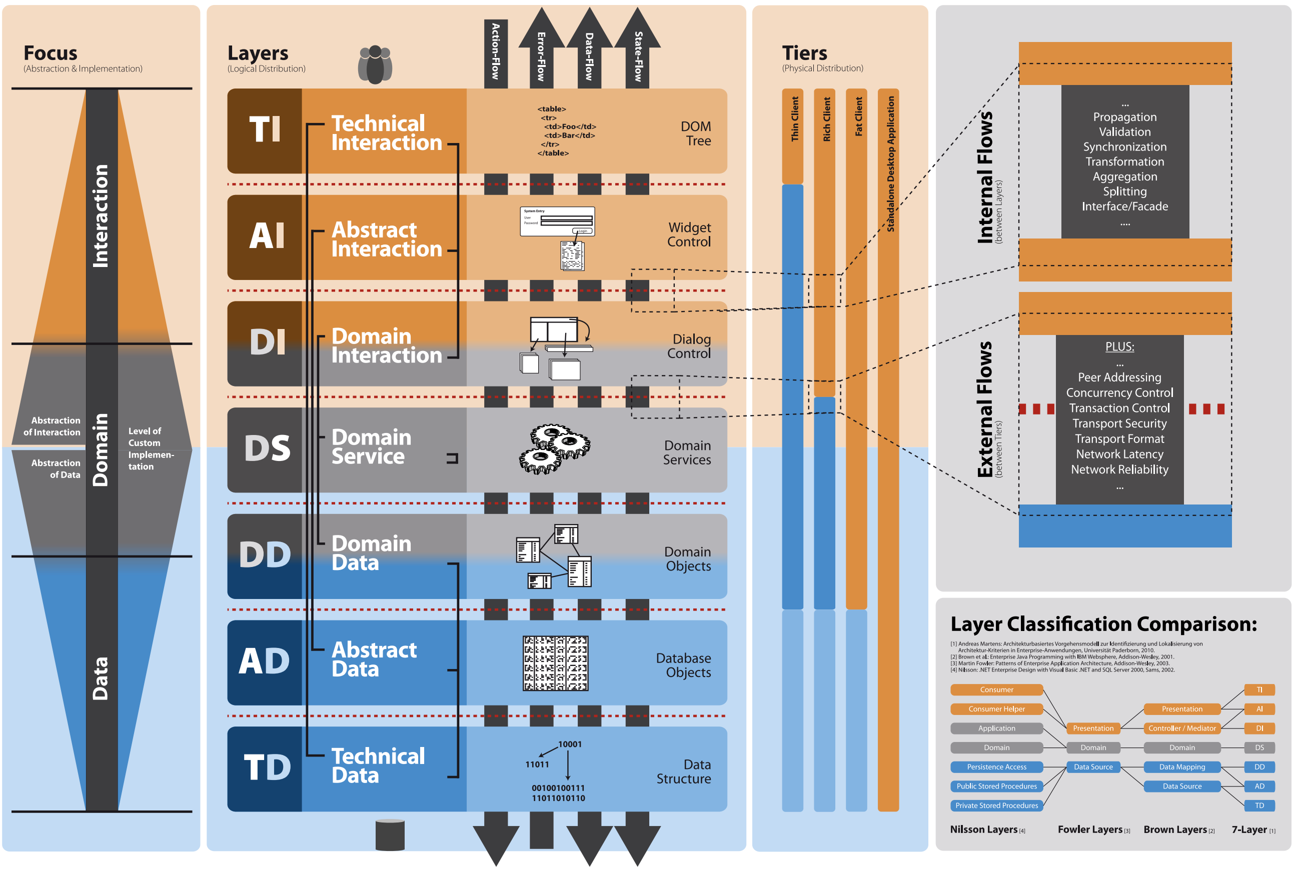
- ? What is the name of the approach in which data is replicated from a master system to many slave systems?

ARCHITECTURE FUNDAMENTALS

Application Reference Architecture

TECHNISCHE UNIVERSITÄT MÜNCHEN

Intellectual Content: Version 1.0.9 (2022-10-09), Authored 2009-2022 by Matthias Brusdeylins, Andreas Martens and Dr. Ralf S. Engelschall
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.



Layer Classification Comparison:

[1] Andreas Martens: Architekturbasiertes Vorgehensmodell zur Identifizierung und Lokalisierung von Architektur-Kriterien in Enterprise-Anwendungen, Universität Paderborn, 2010.
[2] Brown et al.: Enterprise Java Programming with IBM Websphere, Addison-Wesley, 2001.
[3] Martin Fowler: Patterns of Enterprise Application Architecture, Addison-Wesley, 2003.
[4] Nilsson: .NET Enterprise Design with Visual Basic .NET and SQL Server 2000, Sams, 2002.

In an application, it is possible to distinguish 7 logical layers, each grouped in two ways: on the one hand, there are the three sequential layer groups **Technical/ Abstract/Domain Interaction**, **Domain Service** and **Domain/Abstract/Technical Data**, on the other hand, there are the three nested layer groups **Technical Interaction/Data**, **Abstract Interaction/Data** and **Domain Interaction/Service/Data**.

In addition, one can distinguish 4 primary flows in an application: the **Action Flow** consequently runs from top to bottom only because all actions at the top are triggered by the user (or neighboring systems); the **Error Flow** consistently runs only in the opposite direction, i.e., from the bottom to the top, because errors, in the worst case, must be reported to the user; the (domain-specific) **Data Flow** and the (technical) **State Flow** run in both directions because data and states have to be persisted as well as displayed.

The abstraction of Interaction/Data in the layers increases from the top/bottom towards the middle, so most of the functional code of an application is written there. For the upper/lower layers, one usually massively relies on Open Source libraries/frameworks.

If instead of a logical cut (resulting in an **Internal Flow** between the layers) between two layers, one makes a physical cut (which then results in an **External Flow**), i.e., one distributes the application into single programs on different computers, then the resulting architecture is called according to the scope and responsibility of the client.

With **Thin Client**, only the **Technical Interaction** is offloaded to the client, while with **Rich Client** the entire user interface (i.e., all three layers **Technical/Abstract/ Domain Interaction**) is autonomously offloaded to the client (usually as a so-called “HTML5 Single-Page-Application”), with **Fat Client** there is no more associated server at all, and with the **Standalone** application, there is only one single program.

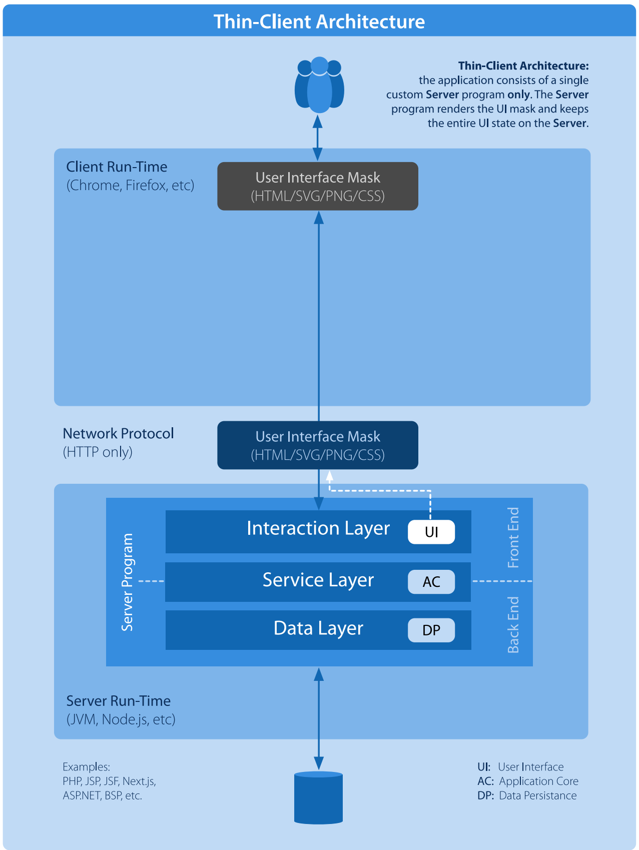
Questions

- ? What is the name of the application architecture in which the entire user interface runs autonomously on the client, while the server only provides purely functional services?
- ? What are the web applications named that implement a **Rich Client** architecture?

ARCHITECTURE
FUNDAMENTALS

Client-Server Architecture

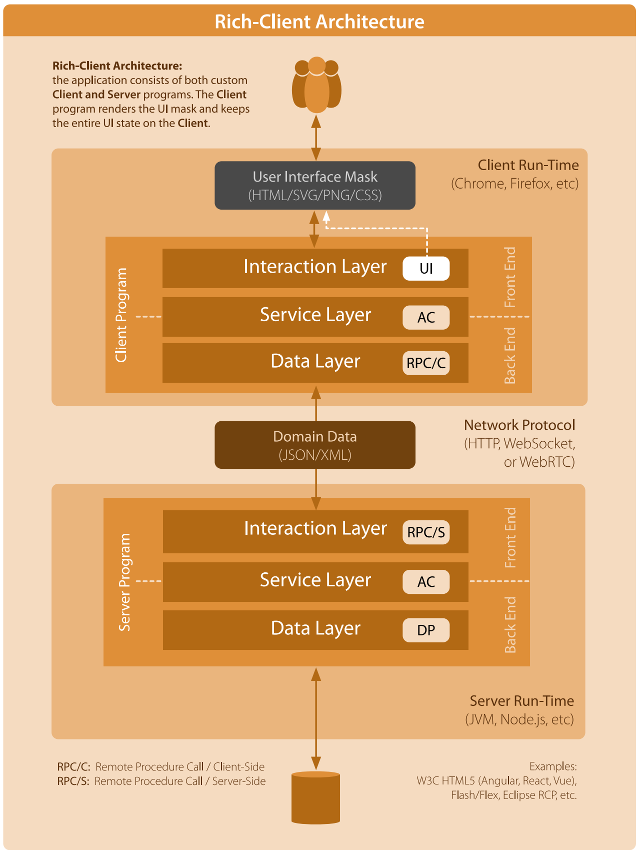
TECHNISCHE
UNIVERSITÄT
MÜNCHEN



Thin-Client Architecture: the application consists of a single custom **Server** program **only**. The **Server** program renders the UI mask and keeps the entire UI state on the **Server**.

Examples:
PHP, JSP, JSF, Next.js,
ASP.NET, BSP, etc.

UI: User Interface
AC: Application Core
DP: Data Persistance



Rich-Client Architecture: the application consists of both custom **Client and Server** programs. The **Client** program renders the UI mask and keeps the entire UI state on the **Client**.

RPC/C: Remote Procedure Call / Client-Side
RPC/S: Remote Procedure Call / Server-Side

Examples:
W3C HTML5 (Angular, React, Vue),
Flash/Flex, Eclipse RCP, etc.

In the **Thin-Client Architecture**, the application consists of a single custom Server program only. This Server program renders the User Interface Mask and keeps the entire state of the User Interface on the Server.

The advantage of this architecture is that the application can be updated very easily. The disadvantage of this architecture is that the user interface reacts sluggishly, and the state of the user interfaces of all clients must be kept on the server, which can make the server a bottleneck.

In the **Rich-Client Architecture**, the application consists of both custom Client and Server programs. The Client program renders the User Interface mask and keeps the entire state of the User Interface on the Client.

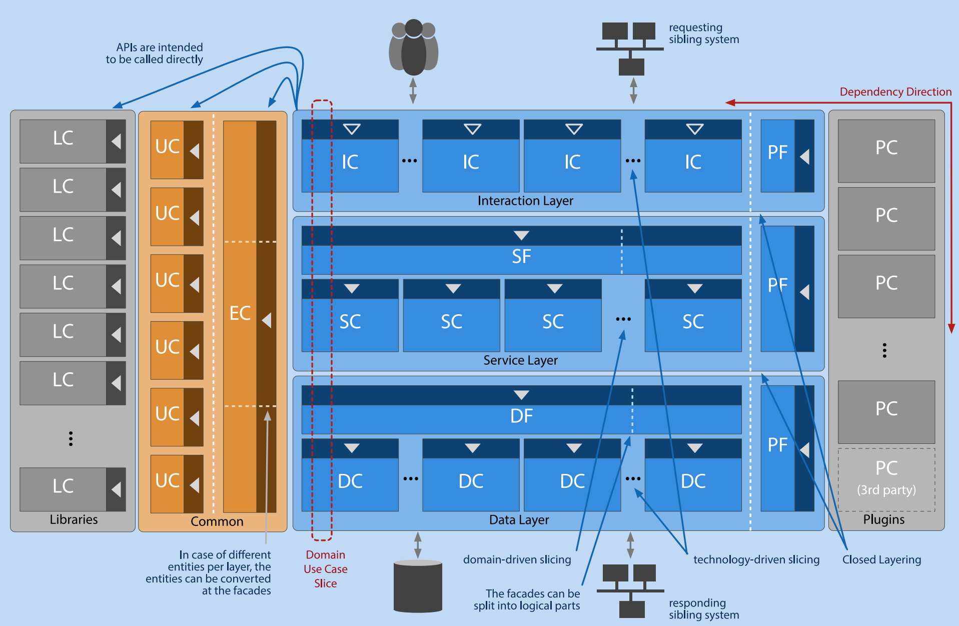
The advantage of this architecture is that the user interface is highly responsive, only domain-specific data has to be exchanged between the client and the server and the server becomes less of a bottleneck. The disadvantage of this architecture is that, if necessary, the client has to be updated explicitly via an installation procedure.

Questions

- With which Client Architecture does the User Interface offer the higher responsiveness?

Information System Architecture

Reference Architecture Blueprint



Component Types:

IC: Interaction Component
SF: Service Facade
SC: Service Component
DF: Data Facade
DC: Data Component

PF: Plugin Facade
PC: Plugin Component
UC: Utility Component
EC: Entity Component
LC: Library Component

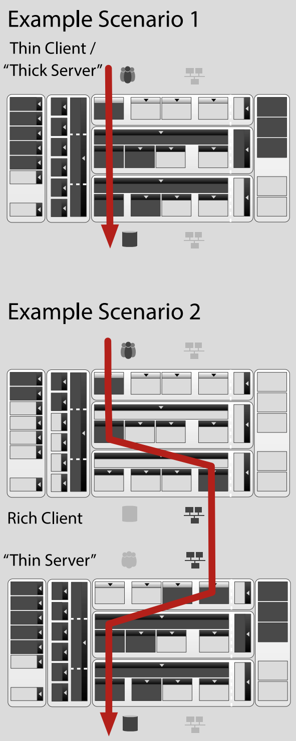
Service Facade (SF) Potential Functionality:
Request Validation, Use-Case Authorization, Process Orchestration (Dispatching), Data Splitting/Aggregation/Conversion, Transaction Handling, Run-Time Use-Case Tracing, Result Caching, etc.

Data Facade (DF) Potential Functionality:
Data Access Authorization, Data Orchestration (Dispatching), Data Splitting/Aggregation/Conversion, Transaction Handling, Run-Time Data Access Tracing, Data Caching, etc.

▽ Input/Output Interface

▼ Application Programming Interface (API)

Blueprint Usage



A (business) Information System usually follows a stringent component-based reference architecture. This is represented "full blown" and can be arbitrarily "slimmed down."

First, this reference architecture consists of 3 substantial Layers: the **Interaction Layer** with the (technically cut) components, which provide the I/O-based interfaces to the user (User Interface) and/or requesting neighboring systems (via Network interface), the **Service Layer** with the (domain-specifically cut) service components (also called Application Core) and the **Data Layer** with the (technically cut) components that provide the connection to the own database and/or neighboring systems to be queried.

Note that the "docking position" of a neighboring system depends on its roles: if it requests, it docks to the Interaction Layer; if it is queried, it docks at the Data Layer. If it happens to have both roles, it docks twice. The other view is that both the user and the database can be understood as special "neighbor systems."

To connect the N **Interaction Components** (IC) with M **Service Components** (SC), a decoupling **Service Facade** (SF) is usually inserted. For the same reason, there is usually also a **Data Facade**.

The Data Model is offloaded to common **Entity Components** (EC). Together with possibly shared code, both live in one **Common Slice**. Libraries and Plugins are also offloaded to separate slices, but there are two major differences: Libraries are passive and provide their functionality to the application via their interfaces. Plugins are active and control the application in that they hook into the application via Service Provider Interfaces (SPI) of the **Plugin Facades**.

In the application, there has to be only exactly one **Dependency Direction** so that the application (in the opposite direction of the dependencies) can be built cleanly. The reference architecture is usually also instantiated twice in order to design both a Rich Client and an associated "Thin Server" from it.

Questions

- With which layer pattern can the **Interaction Components** be decoupled from the **Service Components** in an Information System?

- In which order are the components of an application built?